

Progress Report & Benchmarking Plan

January 16, 2026 – February 27, 2026

Overview & Objective

Over the past six weeks, my primary research focus has been on debugging, stabilizing, and validating the OpenMFDA (Microfluidic Design Automation) flow. The software pipeline is now fully operational within its Dockerized environment, correctly handling everything from high-level design specification to 3D geometry generation (OpenSCAD) and fluidic simulation (Xyce).

With the software platform now stable, the objective is to begin physical benchmarking. I plan to use the OpenMFDA platform to generate microfluidic designs, manufacture physical prints of these devices, and experimentally validate the software's auto-generated geometries and simulation models.

Proposed Benchmarking & Testing Plan

To rigorously benchmark the OpenMFDA software against real-world performance, testing will proceed as follows:

- 1. Automated Design Generation:** Utilize the stabilized `urinalysis_main.py` flow to automatically generate layouts and 3D printable files for test geometries (such as the urinalysis chip).
- 2. Physical Fabrication:** 3D print the auto-generated microfluidic devices to assess the manufacturability of the software's routing algorithms and component library definitions.
- 3. Experimental Validation vs. Simulation:** Run physical fluidic tests on the manufactured prints (e.g., flow rate analysis, pressure testing). We will then directly compare these empirical results against the theoretical data generated by our recompiled Xyce/Verilog-A simulation engine.
- 4. Software Refinement:** Use any physical discrepancies to feedback into the software—refining the Verilog-A component models, OpenSCAD dimensional tolerances, and routing parameters.

OpenMFDA Flow Stabilization (Work Completed)

To prepare the system for this physical benchmarking phase, several critical software roadblocks were diagnosed and resolved:

- **Simulation Engine (Xyce) Execution:** Debugged the Xyce simulation pipeline for the urinalysis flow. This required modifying the system's Dockerfile to inject missing dependencies (`libxml-libxml-perl`, `libgd-perl`, `libtool-bin`), updating library paths, and resolving trace `FileNotFoundError` exceptions related to reading `.prn` simulation outputs.

- **Verilog-A Plugin Compilation:** Rebuilt and recompiled the custom Xyce plugin (MFXyce_0.0.0.so) within the Docker container to restore Verilog-A model compatibility, an essential step for generating correct simulation data to compare against our physical prints.
- **Pipeline Execution (urinalysis_main.py):** Resolved crashes in the main automation script by identifying and installing missing toolchain dependencies (yosys, matplotlib), correcting python return value unpacking, and properly configuring the OPENMFDA_ROOT environment mapping.
- **3D Geometry (OpenSCAD) Generation:** Debugged the 3D rendering pipeline to ensure printable geometries. Fixed missing include dependencies (lef_scad_config.scad), cleaned up duplicate/overlapping variables in myurinalysis_components.scad, and corrected len() validation issues in the routing.scad scripts.
- **OS Compatibility:** Investigated toolchain portability by pulling and assessing GitHub repositories related to “Renner OS Compat”.

Additional Coursework & Software Tasks

- **Python Data Processing:** Implemented a data parsing protocol for collating budget text logs into a structured output compatible with a pie chart visualization script (BudgetPieChart.py).
- **Statistical Analysis (R):** Completed R programming modules including computing rigorous 95% two-sided confidence intervals for dataset means, and adhering to strict script linting and formatting standards.
- **OpenSCAD Scripting:** Implemented route_scripts.py module to handle the generation of 3D microfluidic channel geometries, including the definition of complex channel networks and component placements.